

反復型開発 5巡目

要求

- シナリオトレーサーでブロックをターゲットまで運ぶ

設計

- 現時点の時間をトリガーとするシナリオの切替で実行

実装

unit/Walker.h

- メソッド追加

```
public:
    static const int RIGHT_TURN = -25;
    static const int LEFT_TURN = 25;

    Walker(spikeapi::Motor& leftWheel,
           spikeapi::Motor& rightWheel);

    void init();
    void run();
    void setCommand(float turn);
    void setCommand(float pwm, float turn, float bias);    // 追加
```

- 属性追加、修正

```
private:
    spikeapi::Motor& mLeftWheel;
    spikeapi::Motor& mRightWheel;
    int mPwm;        // 追加
    int mTurn;
    int mBias;       // 追加

    // 以下削除
    // #ifndef MAKE_RASPIKE
    //     const int8_t pwm = 45;
    // #else
    //     const int8_t pwm = 40;
    // #endif
```

unit/Walker.cpp

- コンストラクタの修正

```
/**
 * コンストラクタ
 * @param leftWheel 左モータ
 * @param rightWheel 右モータ
 */
Walker::Walker(spikeapi::Motor& leftWheel,
                spikeapi::Motor& rightWheel)
    : mLeftWheel(leftWheel),
      mRightWheel(rightWheel),
      mPwm(0),      // 追加
      mTurn(0),     // 修正
      mBias(0) {    // 追加
}
```

- 走行するのメソッド修正

```
/**
 * 走行する
 */
void Walker::run() {
    // 左右モータに回転を指示する
    mLeftWheel.setPower(mPwm - mTurn - mBias); // 修正
    mRightWheel.setPower(mPwm + mTurn + mBias); // 修正
}
```

- メソッド追加

```
/**
 * pwm, turn, bias を設定する
 * @param pwm 前進量
 * @param turn 旋回量
 * @param bias バイアス
 */
void Walker::setCommand(float pwm, float turn, float bias) {
    mPwm = pwm;
    mTurn = turn;
    mBias = bias;
}
```

app/LineTracer.h

- 定数追加

```
public:
    static const float Kp;
    static const int DEFAULT_PWM;    // 追加
    static const int DEFAULT_BIAS;   // 追加
```

app/LineTracer.cpp

- 定数宣言追加

```
// 定数宣言
const float LineTracer::Kp = 0.83;
const int LineTracer::DEFAULT_PWM = 45;
const int LineTracer::DEFAULT_BIAS = 0;
```

- ライントレースするのメソッドの修正

```
/**
 * ライントレースする
 */
void LineTracer::run()
{
    if (mIsInitialized == false)
    {
        mWalker->init();
        mIsInitialized = true;
    }

    int diffReflection = mLineMonitor->calDiffReflection();

    // 走行体の操作量を計算する
    float turn = _EDGE * mPidController.calcValue(diffReflection);
    mWalker->setCommand(DEFAULT_PWM, (int)turn, DEFAULT_BIAS);    // 修正

    // 走行を行う
    mWalker->run();
}
```

- 走行体の操作量を計算するのメソッドを修正

```
/**
 * 走行体の操作量を計算する
 * @param diffBrightness ラインから外れた度合い（ライン閾値との差）
 */
```

```
float LineTracer::calcPropValue(int diffBrightness)
{
    float turn = LineTracer::Kp * diffBrightness + LineTracer::DEFAULT_BIAS;    //
    修正

    return turn;
}
```

app/Scene.h

- SceneCommandの追加

```
enum SceneCommands {
    GO_STRAIGHT = 0,    // 直進
    TURN_LEFT,          // 左前ターン
    TURN_RIGHT,         // 右前ターン
    BACKWARD,           // 後退
    TURN_BACK_LEFT,     // 左後ターン
    TURN_BACK_RIGHT,    // 右後ターン
    STOP,               // 停止
    FINISH              // 完全停止
};
```

app/ScenarioTracer.h

- 定数の追加

```
public:
    static const int DEFAULT_PWM;    // 追加
    static const int DEFAULT_TURN;  // 追加
    static const int DEFAULT_BIAS;  // 追加
```

app/ScenarioTracer.cpp

- include 追加、定数定義の追加

```
#include "ScenarioTracer.h"
#include "etroboc_ext.h"    // 追加

const int ScenarioTracer::DEFAULT_PWM = 25;    // 追加
const int ScenarioTracer::DEFAULT_TURN = 25;    // 追加
const int ScenarioTracer::DEFAULT_BIAS = 0;    // 追加
```

- トレースコマンド設定のメソッドを修正

```
/**
 * トレースコマンド設定
 * @param command 走行向き
 */
void ScenarioTracer::setCommand(SceneCommands command) {
    int pwm = 0;
    int turn = -1;
    int bias = 0;

    switch (command)
    {
    case SceneCommands::GO_STRAIGHT:    // 直進の場合
        pwm = DEFAULT_PWM;
        turn = 0;
        bias = DEFAULT_BIAS;
        break;
    case SceneCommands::TURN_LEFT:      // 左前ターン
        pwm = DEFAULT_PWM;
        turn = DEFAULT_TURN;
        bias = DEFAULT_BIAS;
        break;
    case SceneCommands::TURN_RIGHT:     // 右前ターン
        pwm = DEFAULT_PWM;
        turn = - DEFAULT_TURN;
        bias = DEFAULT_BIAS;
        break;
    case SceneCommands::BACKWARD:       // 後退
        pwm = - DEFAULT_PWM;
        turn = 0;
        bias = DEFAULT_BIAS;
        break;
    case SceneCommands::TURN_BACK_LEFT: // 左後ターン
        pwm = - DEFAULT_PWM;
        turn = DEFAULT_TURN;
        bias = DEFAULT_BIAS;
        break;
    case SceneCommands::TURN_BACK_RIGHT: // 右後ターン
        pwm = - DEFAULT_PWM;
        turn = - DEFAULT_TURN;
        bias = DEFAULT_BIAS;
        break;
    case SceneCommands::STOP:           // 停止
        pwm = 0;
        turn = 0;
        bias = 0;
        break;
    case SceneCommands::FINISH:         // 完全停止
        pwm = 0;
        turn = 0;
        bias = 0;
        ETRoboc_notifyCompletedToSimulator();
    }
```

```

        break;
    default:
        break;
    }

    // if (command == TURN_LEFT) {
    //     turn = Walker::LEFT_TURN;
    // } else if (command == TURN_RIGHT) {
    //     turn = Walker::RIGHT_TURN;
    // }

    // mWalker->setCommand(turn);

    mWalker->setCommand(pwm, turn, bias);
}

```

app.cpp

- scene object の修正

```

// scene object
static Scene gScenes[] = {
    { TURN_LEFT,      895 * 1000, 0 }, // 左旋回0.895秒
    { GO_STRAIGHT, 12 * 1000 * 1000, 0 }, // 直進12秒
    { STOP,          1 * 1000 * 1000, 0 }, // 停止1秒
    { BACKWARD,      3 * 1000 * 1000, 0 }, // 後退3秒
    { TURN_LEFT,      450 * 1000, 0 }, // 左前ターン0.45秒
    { GO_STRAIGHT,  8 * 1000 * 1000, 0 }, // 前進8秒
    { TURN_LEFT,      580 * 1000, 0 }, // 左前ターン0.58秒
    { GO_STRAIGHT,  6 * 1000 * 1000, 0 }, // 前進6秒
    { STOP,          1 * 1000 * 1000, 0 }, // 停止1秒
    { FINISH,        10 * 1000 * 1000, 0 }, // 完全停止10秒
};

```

検証

- 確認事項
 - 左コースで120秒以内に完走できるか確認
 - シナリオレースの微調整を行う（時間調整）